

Agent Acceleration in a Stable Environment

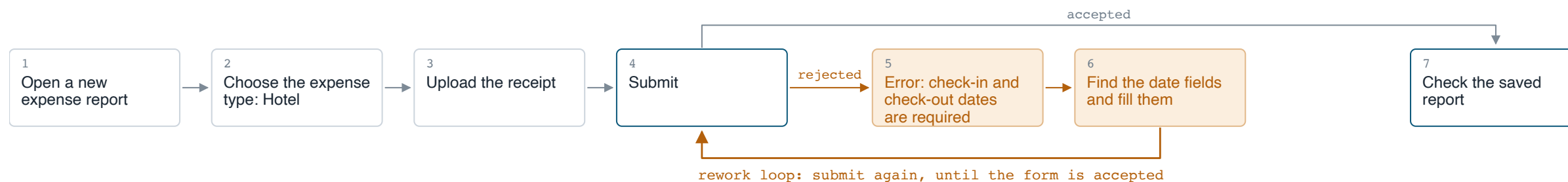
Learning from repeated tasks to make later ones faster and cheaper

October 2026

Example: a web agent that files expense reports

Our running example: a web agent files expense reports; each task is a new report in the same system. Not knowing one rule of the form, that a hotel expense needs check-in and check-out dates, sends the agent into a rework loop.

ONE HOTEL REPORT, BEFORE THE AGENT KNOWS THE RULE



WITHOUT LEARNING

The loop is paid again on every hotel report

- Its extra model calls and seconds are part of $\bar{v}(m)$, for each of the n reports

LEARNING THE RULE ONCE

The rule is learned once, at cost C_{learn} ; later reports skip the loop

- $\bar{v}(m') < \bar{v}(m)$; it pays off after n^* reports (page 1)

IN THIS EXAMPLE

- **Environment:** the expense system, its forms, the fields each expense type requires, the actions it allows
- **Run record:** the steps above, with the error message, and the time and cost of each step

WHAT LEARNING MAY CHANGE HERE (MODEL WEIGHTS FIXED)

- **Prompt:** an instruction read on every call: fill the check-in and check-out dates before submitting a hotel expense
- **Memory:** a note read only when relevant: Hotel → check-in and check-out dates required
- **Skill or tool:** a procedure the model calls when it needs it: instructions plus a script that fills every field a hotel expense requires
- **Control code:** the harness's own logic, run whatever the model decides: check the required fields before every submit

SCOPE this example: expense reports on a website → any workflow repeated in one software environment → websites · desktop applications · mobile apps · tool APIs

Five difficulties, grouped by the cost they affect

COST PER TASK AFTER LEARNING · $\bar{v}(m')$

1 · Fewer steps do not always save time

BEFORE

AFTER

10.7 steps · 215 s

10.2 steps · 395 s

one step

extra reading and thinking in each step

length = seconds per task · per step 20.1 → 38.7 s (calc.)

Each step got longer: judge a report by its seconds, not its steps

Drawn from WebCoach (Liu et al., 2025). Also: ReasoningBank (Ouyang et al., 2026), fewer steps, more tokens

2 · Whether a check is needed is not known in advance

Choose type: Hotel

check the form

fill dates

submit

✓

schematic

one step more, always right

skip: as last time

submit

✓ rule still holds

✗ it changed: rework

Skipping a check saves a step only while the rule still holds

Seen in: DRAFT (Qu et al., 2025), learns tool conditions by trying · SKILL.nb (Hattami et al., 2026), checks before reuse

3 · A trade-off between generalization and cost per use

generalizes ↑

cheap and general?

note in every prompt

note in memory

skill: steps + script

replayed script

schematic

cost per use →

Seen in: Metis (Dai et al., 2026), keeps both text and code · ActionEngine (Zhong et al., 2026), turns a site map into programs

COST OF LEARNING · C_{learn}

4 · Finding the cause behind an error message

Error: over the meal limit

possible causes, of different kinds, checked one by one

reasoning wrong type?

rule limit changed?

interaction wrong field?

tool amount misread?

...?

schematic

the cause may be found late, or not at all

cost of checking so far →

Each possible cause costs a check; the right one may come late, or never

Seen in: HarnessFix (Chen et al., 2026), finds the faulty step · ESPO (L. Liu et al., 2026), sorts errors first

5 · Learning and testing can cost more than they save

total cost ↑

learning and testing are paid first

break-even n^*

break-even, more testing

repair after the site changes

no learning more testing with learning

schematic

number of reports filed →

Learning pays off only if enough reports share its cost

Seen in: AI Agents That Matter (Kapoor et al., 2025), late break-even · SICA (Robeyns et al., 2025), large bill, small saving

REFERENCES

• Chen, M., Wang, J., Liu, Z., et al. (2026). From Failed Trajectories to Reliable LLM Agents: Diagnosing and Repairing Harness Flaws. *arXiv preprint*.

• Dai, Z., He, S., Li, H., et al. (2026). Metis: Bridging Text and Code Memory for Self-Evolving Agents. *arXiv preprint*.

• Hattami, A. E., Chapados, N., & Pal, C. (2026). SKILL.nb: Selective Formalization and Gated Execution for Durable Agent Workflows. *FAGEN@ICML 2026 Poster (ICML 2026 workshop, venueid ICML.cc/2026/Workshop/FAGEN)*.

• Kapoor, S., Stroebel, B., Siegel, Z. S., et al. (2025). AI Agents That Matter. *Transactions on Machine Learning Research*.

• Liu, G., Geng, S., Li, S., et al. (2025). WebCoach: Self-Evolving Web Agents with Cross-Session Memory Guidance. *ICLR 2026 Workshop on Memory for LLM-Based Agentic Systems (MemAgents)*.

• Liu, L., Tang, P., Singh, K. Y., et al. (2026). ESPO: Error-Structured Prompt Optimization via Diagnose, Diversify, and Stabilize. *EMNLP 2026*.

• Ouyang, S., Yan, J., Hsu, I. H., et al. (2026). ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory. *ICLR 2026*.

• Qu, C., Dai, S., Wei, X., et al. (2025). From Exploration to Mastery: Enabling LLMs to Master Tools via Self-Driven Interactions. *ICLR 2025 (International Conference on Learning Representations 2025)*.

• Robeyns, M., Szummer, M., & Aitchison, L. (2025). A Self-Improving Coding Agent. *ICLR 2025 Workshop 'Self-Improving Foundation Models Without Human Supervision'*.

• Zhong, H., Faisal, F., França, L., et al. (2026). ActionEngine: From Reactive to Programmatic Web Agents via State Machine Memory. *arXiv preprint*.

D · Evidence behind the five difficulties (1/2)

WORK	VENUE	WHAT IT CHANGES	WHERE IT WAS TESTED	EFFECT, WITH CONDITIONS	TIME, MONEY, LEARNING COST	MARKS
WebCoach (Liu et al., 2025)	ICLR 2026 MemAgents workshop	External trajectory summary, additional Coach injects suggestions to action model	WebVoyager 643 real-time web tasks; dynamic library accumulates same-stream same-task but excludes same-task-ID retrieval	Skywork-38B + Qwen3-8B Coach: 0.614 vs 0.473; Qwen-VL-7B main table 0.311 vs 0.328	Skywork average per question 215 → 395 s, actions 10.7 → 10.2; time includes extra inference; no token cost measurement	● ○ ●
ReasoningBank (Ouyang et al., 2026)	ICLR 2026	Text experience pool extracted from success and failure trajectories	WebArena, SWE-bench; online single-task stream	WebArena Gemini-2.5-flash success rate 40.5 → 48.8; Shopping judge accuracy 72.7%	WebArena average ReAct steps 9.7 → 8.3; another table total tokens 50,847.4 → 53,054.5, baseline foundation not specified; time and cost not measured	● ○ ●
GenericAgent (Liang et al., 2026)	arXiv	SOP, script, hierarchical memory; fixed tool interface	Multiple models; local task, web, repeated download and research tasks	Same GPT-5.4 repeated download 102 seconds → ~66 seconds; not held-out; five long tasks success 100%, vs Claude Code 100%, OpenClaw 80%	Repeated download tokens 200,439 → 100,000; five long tasks 220.8 s vs OpenClaw 183.1 s; evolution cost not separated	● ● ●
EchoPath (Zhao et al., 2026)	arXiv	Parameterized GUI trajectory memory, precondition and lifecycle	OSWorld-Verified 159 task with established memory; same-task second try change resolution, codex-gpt-5.5-medium	91.2% vs Synapse 91.8%	Median tokens 20,370 vs 586,386; time 127.5 s vs 315.7 s; first build median ~572k tokens/4.5 min counted success-only	● ● ○
MobileGPT (Lee et al., 2024)	ACM MobiCom 2024	Per-app page-subtask graph and action sequence	Mobile apps; cold start then warm start same task with parameter change	GPT-4-turbo; with human repair ablation warm start 98.75%; main table without human repair cannot mix	Human-repair path warm start vs same prompt Derive-only: latency -62.5%, cost -68.8%; offline explore per app 10 to 15 minutes, total \$10.78	● ● ●
ActionEngine (Zhong et al., 2026)	arXiv	State-action graph, locator and descriptor, online general-purpose repair write-back	WebArena 655 tasks, Claude Opus 4.6; template warm-up then eval, patcher off	91.2%; no warm-up graph 73.1%	27 s vs 87 s; \$0.05 vs \$0.40, both per-task relative Claude Code; no explore/warm-up; break-even 39- 101 tasks and missing warm-up	● ● ●
Are Online Skill and Memory Modules Always Worth Their Tokens? (Hajimiri et al., 2026)	EMNLP 2026 main conference (listed in the programme)	Control AWM, ASI, ReasoningBank vs plain executor budget config	WebArena four domain, Gemini 3 Flash, three run	Vanilla-IB success 44.78%, 73.6K total tokens/question, that model four-config best simultaneously	Token only; plain 15 step+pruning, enhance 10 step, near-equal budget	● ● ●
ClawTrace / CostCraft (Yuan et al., 2026)	ACM CAIS 2026 Agent Skills workshop, non- archival	Trajectory-summary exported skill document, frozen model	SpreadsheetBench 30 held-out, OpenClaw + gpt-5.4; also migrate SkillsBench	Remove delete patch after two-type sum degrade 5 → 14; migrate post cost median not decrease	Execution cost with cache; SkillsBench median \$0.143 vs \$0.144; extract cost not reported	● ○ ●

Marks in the last column, in order: time or money of runs measured · learning cost measured · tested on unseen tasks; ● yes, ○ partly, ○ no (initial judgment, calc.; Appendix A1).

D · Evidence behind the five difficulties (2/2)

WORK	VENUE	WHAT IT CHANGES	WHERE IT WAS TESTED	EFFECT, WITH CONDITIONS	TIME, MONEY, LEARNING COST	MARKS
SEDM (Xu et al., 2025)	NeurIPS 2025 SEA workshop	Injectable memory fragments with utility weight and source	LoCoMo, FEVER, HotpotQA; independent build vs evaluation not clear	GPT-4o-mini FEVER: 66 vs G-Memory 62, no memory 57; HotpotQA 39 / 38 / 34	FEVER prompt tokens 2.47M vs G-Memory 3.62M, no memory 1.65M; replay cost and time cost not measured	● ○ ●
HarnessFix (Chen et al., 2026)	arXiv	Prompts, tool description, orchestration, config, adapter and validation script	GAIA, SWE-bench Verified, AppWorld, Terminal-Bench 2.0 Verified; GPT-5 mini	Initial system to patched completion +6.3 to +18.4 percentage points; mean of three runs	AppWorld offline patch 37.2 million tokens; execution-phase time/cost not reported	○ ● ●
ESPO (L. Liu et al., 2026)	EMNLP 2026 Findings (listed in the programme)	Natural-language module prompt	Seven text benchmark, each 500 held-out; Claude Sonnet 4.5	Main optimize single average 74.67% vs GEPA 70.91%; prompt 1,004 vs 1,878 character	PUPA optimize ~3.4M tokens, 2h 13m; no deployment acceleration measured	○ ● ●
GEPA (Agrawal et al., 2026)	ICLR 2026	Module prompts only, model frozen	QA, instructions, privacy rewriting, math; train validation test separated	Qwen3 8B AIME-2025: 32.00 vs MIPROv2 20.00, GRPO 38.00; overall advantage ≠ every task wins	Per-task optimization rollout total 6871 / 3593 / 7051 / 2426 / 1839 / 1839; GRPO per item 24000; not model calls, time or cost	○ ● ●
SICA (Robeyns et al., 2025)	ICLR 2025 Self-Improving Foundation Models Without Human Supervision workshop	Self-complete Python code, tools and oversight mechanism	Sonnet3.5(v2), o3-mini; fixed coding subset and synthetic-file task	SWE peak 17% → 53%, final accuracy 0.51; same baseline for agent selection, not held-out	Cross-four-benchmark full-attempt per-question average: final \$1.91 → \$1.70, 130.2 → 114.5 s; tokens 0.24 → 0.30 M; full cycle ~\$7,000	● ● ○
ADAS (Hu et al., 2025)	ICLR 2025 main conference	Fixed small framework forward() program	GPT-4o search, GPT-3.5 execution; ARC, QA reasoning held-out test	DROP F1 79.4 vs 65.8; MGSM 53.4 vs 39.0, each strongest hand-written baseline	One search + eval ~500 USD (ARC), 300 USD (reasoning); deployment per-question not measured	○ ● ●
Group-Evolving Agents / GEA (Weng et al., 2026)	arXiv	Multiple parent agents share trajectories then modify framework code	SWE-bench/Polyglot; Haiku4.5 then Sonnet4.5, o1 reflection	Report SWE 71.0%, Polyglot 88.3%, but latter inconsistent with claimed 50-question held-out denominator; not use as generalization-gain evidence	Full cycle estimate USD 13,000 / USD 1,500; no per-question post-evolution cost change	○ ● ●

Marks in the last column, in order: time or money of runs measured · learning cost measured · tested on unseen tasks; ● yes, ○ partly, ○ no (initial judgment, calc.; Appendix A1).